

集合関数 ゼータ変換 メビウス変換 AND 畳み込み OR 畳み込み

ゼータ・メビウス変換（部分集合関係）

1. 概要

本講座では、部分集合・上位集合関係に関する**ゼータ変換**、**メビウス変換**について解説します。ゼータ変換・メビウス変換はより一般に順序集合に対して定義されるものですが、本講座で扱うのはその特殊ケースということになります。

2. 前提知識

$\{0, 1, \dots, n-1\}$ の部分集合と、 2^n 未満の非負整数の対応（ビットマスク）。

3. 部分集合に関するゼータ変換

n を非負整数とします。本講座では、 $\{0, 1, \dots, n-1\}$ の部分集合を、ビットマスクに対応させ、 2^n 未満の非負整数として表します。

3.1. 定義

本講座では主として、整数の加法の場合を扱います。整数の加法以外の演算への一般化については第 7 節で簡単に触れます。

【定義 1】

整数列 $A = (A_0, A_1, \dots, A_{2^n-1})$ に対して、次の式によって定まる列 $B = (B_0, B_1, \dots, B_{2^n-1})$ を、 A の**部分集合に関するゼータ変換** (subset zeta transform) や、SOS (Sum Over Subsets) という。

$$B_T = \sum_{S \subseteq T} A_S.$$

例えば $A = (3, 1, 4, 1, 5, 9, 2, 6)$ の部分集合に関するゼータ変換は、 $B = (3, 4, 7, 9, 8, 18, 14, 31)$ となります。

- $B_0 = A_0 = 3.$
- $B_1 = A_0 + A_1 = 4.$
- $B_2 = A_0 + A_2 = 7.$
- $B_3 = A_0 + A_1 + A_2 + A_3 = 9.$
- $B_4 = A_0 + A_4 = 8.$
- $B_5 = A_0 + A_1 + A_4 + A_5 = 18.$
- $B_6 = A_0 + A_2 + A_4 + A_6 = 14.$
- $B_7 = A_0 + A_1 + A_2 + A_3 + A_4 + A_5 + A_6 + A_7 = 31.$

3.2. $O(4^n)$ 時間アルゴリズム

ゼータ変換は定義通りの計算式によって、 $O(4^n)$ 時間で計算することが可能です。例えば以下のような実装が考えられます。

```
vector<int> subset_zeta(int n, vc<int> A) {
    vector<int> B(1 << n);
    for (int T = 0; T < (1 << n); ++T) {
        for (int S = 0; S < (1 << n); ++S) {
            if ((S & T) == S) { // S が T の部分集合であるかどうかを確認する
                B[T] += A[S];
            }
        }
    }
}
```

```
return B;
}
```

3.3. $O(3^n)$ 時間アルゴリズム

やはり定義通りの計算式を用いた、 $O(3^n)$ 時間のアルゴリズムがあります。ただし集合 S を列挙して T の部分集合であるか否かを判定するのではなく、ビット演算のテクニックを使って直接 T の部分集合を列挙するようにします。

```
vector<int> subset_zeta(int n, vc<int> A) {
    vector<int> B(1 << n);
    for (int T = 0; T < (1 << n); ++T) {
        for (int S = T;; S = (S - 1) & T) { // T の部分集合を列挙する for ループ
            B[T] += A[S];
            if (S == 0) break;
        }
    }
    return B;
}
```

このアルゴリズムの計算量は、次の補題から分かります。

【補題 2】

$S \subseteq T \subseteq \{0, 1, \dots, n-1\}$ となる集合の組 (S, T) は 3^n 個である。

組 (S, T) を作ることは、各 $i = 0, 1, \dots, n-1$ に対して次の 3 通りのいずれかを選ぶことと対応します。

- $i \notin S$ かつ $i \notin T$,
- $i \notin S$ かつ $i \in T$,
- $i \in S$ かつ $i \in T$.

したがってその方法の総数は 3^n です。 ■

3.4. $O(n \cdot 2^n)$ 時間アルゴリズム (高速ゼータ変換)

2 進法表記 $T = t_{n-1} \cdots t_1 t_0$, $S = s_{n-1} \cdots s_1 s_0$ を考えます. このとき, B_T に A_S を加えるのは, 次が成り立つ場合です.

- T' を, $t_i = 0$ ならばそのまま, $t_i = 1$ ならばワイルドカード $*$ に置き換えた文字列とする.
- B_T は, S が T' に**マッチする**ような S に対する A_S の和である.

ただし, $0, 1, *$ からなる文字列 (**パターン文字列**) $p_{n-1} \cdots p_1 p_0$ に, $0, 1$ からなる文字列 $s_{n-1} \cdots s_1 s_0$ が**マッチする**とは, 以下の条件を満たすこととします.

- $p_i = 0$ ならば $s_i = 0$,
- $p_i = 1$ ならば $s_i = 1$,
- $p_i = *$ ならば s_i は 0 でも 1 でもよい.

T	T (2 進法表記)	T'	S (2 進法表記)	S
0	000	000	000	0
1	001	00*	000, 001	0, 1
2	010	0*0	000, 010	0, 2
3	011	0**	000, 001, 010, 011	0, 1, 2, 3
4	100	*00	000, 100	0, 4
5	101	*0*	000, 001, 100, 101	0, 1, 4, 5
6	110	**0	000, 010, 100, 110	0, 2, 4, 6
7	111	***	000, 001, 010, 011, 100, 101, 110, 111	0, 1, 2, 3, 4, 5, 6, 7

ゼータ変換を行うには, T の 2 進法表記からはじめて, パターン文字列の 1 をすべて $*$ に置き換えるという処理をすればよいことが分かります. $i = 0, 1, 2, \dots$ の順に, パターン文字列の i 番目の 1 を $*$ に置き換えるという処理をすることで, 次のアルゴリズムが得られます.

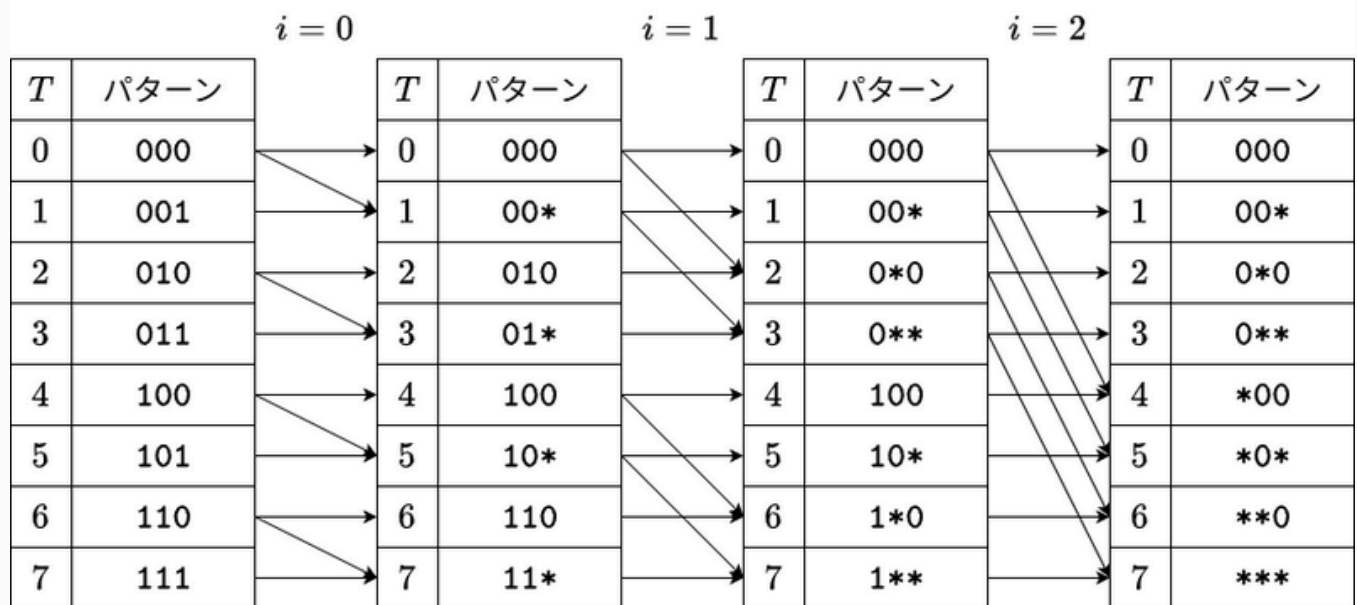
```
vector<int> subset_zeta(int n, vector<int> A) {
    for (int i = 0; i < n; ++i) {
        // i-th bit の '1' を '*' に変更する
        for (int T = 0; T < (1 << n); ++T) {
            if ((T >> i) & 1) {
```

```

        A[T] += A[T ^ (1 << i)];
    }
}
return A;
}

```

時間計算量は $O(n \cdot 2^n)$ 時間です。このアルゴリズムは部分集合に関する**高速ゼータ変換** (fast zeta transform, FZT) と呼ばれます。



なおよりパフォーマンスを良くしたい場合には、「すべての (i, T) を反復して if 文により計算を行うかどうかを判定する」よりも、「直接計算を行うべき (i, T) のみを反復する」ようにした方がパフォーマンスが良いです。この場合、区間 $[2k \cdot 2^i, (2k + 1) \cdot 2^i)$ の部分を区間 $[(2k + 1) \cdot 2^i, (2k + 2) \cdot 2^i)$ の部分に足すというような計算になります。

```

vector<int> subset_zeta(int n, vector<int> A) {
    for (int i = 0; i < n; ++i) {
        int b = 1 << i;
        for (int l = 0, r = b; l < (1 << n); l += 2 * b, r += 2 * b) {
            for (int T = l; T < r; ++T) A[T + b] += A[T];
        }
    }
    return A;
}

```

4. 部分集合に関するメビウス変換

4.1. 定義

ゼータ変換の逆変換をメビウス変換といいます.

【定義 3】

整数列 $B = (B_0, B_1, \dots, B_{2^n-1})$ に対して, 次の式が成り立つような列 $A = (A_0, A_1, \dots, A_{2^n-1})$ を, B の部分集合に関するメビウス変換 (subset Möbius transform) という.

$$B_T = \sum_{S \subseteq T} A_S.$$

なお包除原理より,

$$A_S = \sum_{T \subseteq S} (-1)^{|S|-|T|} B_T$$

が成り立ちますが, 本講座においてはこのことを理解している必要はありません.

4.2. $O(n \cdot 2^n)$ 時間アルゴリズム (高速メビウス変換)

メビウス変換はゼータ変換の逆変換なので, 高速ゼータ変換のアルゴリズムをそのまま逆順に遡ることで, $O(n \cdot 2^n)$ 時間でのメビウス変換のアルゴリズム (高速メビウス変換, fast Möbius transform, FMT) が得られます.

次の実装は, 上で解説した高速ゼータ変換のアルゴリズムが, 完全に逆順に計算が行われるように実装したものです. (i, T) が逆順に走査されることに注意してください.

```
vector<int> subset_mobius(int n, vector<int> A) {
    for (int i = n - 1; i >= 0; --i) {
        for (int T = (1 << n) - 1; T >= 0; --T) {
            if ((T >> i) & 1) {
                A[T] -= A[T ^ (1 << i)];
            }
        }
    }
}
```

```

    }
}
return A;
}

```

とはいえ、 i のループ順つまりどのビットから順にワイルドカードに置き換えるかを入れ替えても問題ありませんし、 T に関するループ順もアルゴリズムに影響しません。よって、次のようなループ順で実装をする方が標準的です。

```

vector<int> subset_mobius(int n, vector<int> A) {
    for (int i = 0; i < n; ++i) {
        for (int T = 0; T < (1 << n); ++T) {
            if ((T >> i) & 1) {
                A[T] -= A[T ^ (1 << i)];
            }
        }
    }
    return A;
}

```

5. 上位集合に関するゼータ変換・メビウス変換

逆向きの包含関係について同様の変換を考えることもあります。

【定義 4】

整数列 $A = (A_0, A_1, \dots, A_{2^n-1})$ に対して、次の式によって定まる列 $B = (B_0, B_1, \dots, B_{2^n-1})$ を、 A の上位集合に関するゼータ変換 (superset zeta transform) という。

$$B_T = \sum_{S \supseteq T} A_S.$$

逆に、 B に対して上の式が成り立つような A を、 B の上位集合に関するメビウス変換 (superset Möbius transform) という。

アルゴリズムも、部分集合に関するゼータ変換・メビウス変換とほとんど同じです（パターン文字列の 0 を * に置き換えていきます）。これらは $O(n \cdot 2^n)$ 時間で計算できます。

6. OR 畳み込み, AND 畳み込み

ゼータ変換, メビウス変換の少し非自明な用途として, OR 畳み込み, AND 畳み込みについて説明します。

6.1. OR 畳み込み

【問題 5】(OR 畳み込み)

整数列 $A = (A_0, A_1, \dots, A_{2^n-1})$ および $B = (B_0, B_1, \dots, B_{2^n-1})$ が与えられます。次の式によって定まる整数列 $C = (C_0, C_1, \dots, C_{2^n-1})$ を求めてください。

$$C_U = \sum_{S \cup T = U} A_S B_T.$$

$S \cup T = U$ という条件が厄介です。一方で, $S \cup T \subseteq U$ という条件は扱いやすく,

$$S \cup T \subseteq U \iff S \subseteq U \text{ かつ } T \subseteq U$$

という S, T について独立な形に書くことができます。そこで計算しやすい値として

$$D_U = \sum_{S \cup T \subseteq U} A_S B_T.$$

を考えると, これは

$$D_U = \left(\sum_{S \subseteq U} A_S \right) \cdot \left(\sum_{T \subseteq U} B_T \right)$$

のように計算できます。この値と, 求めたいもの C_U の関係を考えてみましょう。すると,

$$D_U = \sum_{U' \subseteq U} C_{U'}$$

が成り立つこと、つまり D は C の部分集合に関するゼータ変換であることが分かります。言い換えれば、 C は D の部分集合に関するメビウス変換です。

以上をまとめると、【問題 5】は次のようなアルゴリズムで解けることが分かります。

```
vector<int> or_convolution(int n, vector<int> A, vector<int> B){
    vector<int> ZA = subset_zeta(n, A);
    vector<int> ZB = subset_zeta(n, B);
    vector<int> D(1 << n);
    for(int u = 0; u < (1<<n); ++u) D[u] = ZA[u] * ZB[u];
    vector<int> C = subset_mobius(n, D);
    return C;
}
```

計算量は $O(n \cdot 2^n)$ 時間です。

6.2. AND 畳み込み

AND 畳み込みもほとんど同じです。

【問題 6】(AND 畳み込み)

整数列 $A = (A_0, A_1, \dots, A_{2^n-1})$ および $B = (B_0, B_1, \dots, B_{2^n-1})$ が与えられます。次の式によって定まる整数列 $C = (C_0, C_1, \dots, C_{2^n-1})$ を求めてください。

$$C_U = \sum_{S \cap T = U} A_S B_T.$$

詳細は省略しますが、次のようなアルゴリズムで $O(n \cdot 2^n)$ 時間で計算できます。

```
vector<int> and_convolution(int n, vector<int> A, vector<int> B){
    vector<int> ZA = superset_zeta(n, A);
    vector<int> ZB = superset_zeta(n, B);
    vector<int> D(1 << n);
    for(int u = 0; u < (1<<n); ++u) D[u] = ZA[u] * ZB[u];
    vector<int> C = superset_mobius(n, D);
}
```

```
return C;  
}
```

6.3. 実装例

Library Checker, [Bitwise And Convolution](#) への提出です.

- <https://judge.yosupo.jp/submission/354822>

7. 演算の一般化

高速ゼータ変換は, 可換モノイドに一般化できます. 高速メビウス変換は, 可換群に一般化できます. 本講座の OR 畳み込み, AND 畳み込みは環に一般化できます.

これらの用語の意味については, [代数構造用語集](#) を参照してください.

8. 関連問題

1. <https://cses.fi/problemset/task/1654>
2. https://judge.yosupo.jp/problem/bitwise_and_convolution
3. <https://codeforces.com/contest/383/problem/E>
4. https://atcoder.jp/contests/ndpc/tasks/ndpc2026_m
5. <https://codeforces.com/contest/165/problem/E>
6. https://atcoder.jp/contests/arc100/tasks/arc100_c
7. https://atcoder.jp/contests/abc423/tasks/abc423_f
8. <https://cses.fi/problemset/task/3141>
9. <https://codeforces.com/problemset/problem/449/D>
10. <https://codeforces.com/contest/76/problem/C>
11. <https://yukicoder.me/problems/no/2669>
12. https://atcoder.jp/contests/joi2018ho/tasks/joi2018ho_e
13. <https://yukicoder.me/problems/no/1503>

9. まとめ

本講座では、部分集合・上位集合関係に関するゼータ変換、メビウス変換を $O(n \cdot 2^n)$ 時間で行う方法について解説しました。また、その応用として、AND 畳み込み、OR 畳み込みが $O(n \cdot 2^n)$ 時間で行えることを確認しました。

最後に、本講座で扱った内容の一般化や発展的な内容について簡単に触れておきます（いずれも Algorithm Lectures で扱う予定です）。

ゼータ変換、メビウス変換はより一般的には、**順序集合** (poset) 上の関数に対する操作として定義することもできて、本講座の内容は、その部分集合関係・上位集合関係という具体的な場合と見なすことができます。また、AND 畳み込みや OR 畳み込みは**束** (lattice) へと一般化されます。競技プログラミングでは、1 次元、2 次元の累積和や、約数倍数関係に関するものが有名です。

また、高速ゼータ変換、高速メビウス変換の「各 i について順に、 i 番目のビットに関する変換をする」というアルゴリズムは、線形代数における行列のクロネッカー積によって理解することもできて、やはり競技プログラミングでも複数の場面で見かける考え方です。

OR 畳み込みについては、 $S \cup T = U$ という条件を、 $S \amalg T = U$ （つまり $S \cup T = U$ かつ $S \cap T = \emptyset$ ）に置き換えた版である **Subset Convolution**（部分集合畳み込み）も重要で、OR 畳み込みと非常に近いアルゴリズムによって計算されます。

10. 参考文献

1. USACO Guide. Sum over Subsets DP. <https://usaco.guide/plt/dp-sos>. (閲覧日: 2026-04-01).
2. naoya_t. 高速ゼータ変換／高速メビウス変換.
<https://naoyat.hatenablog.jp/entry/zeta-moebius>. (閲覧日: 2026-04-01).
3. convexineq. ゼータ変換・メビウス変換を理解する.
<https://qiita.com/convexineq/items/afc84dfb9ee4ec4a67d5>. (閲覧日: 2026-04-01).

4. 37zigen. 数え上げのための半順序集合入門. URL : <https://37zigen.com/半順序集合入門>. (2026-06-11 現在リンク切れ).
5. Wikipedia. 順序集合. <https://ja.wikipedia.org/wiki/順序集合>. (閲覧日: 2026-04-01).
6. Wikipedia. 隣接代数 (順序理論). [https://ja.wikipedia.org/wiki/隣接代数_\(順序理論\)](https://ja.wikipedia.org/wiki/隣接代数_(順序理論)). (閲覧日: 2026-04-01).

トップページ : [AtCoder Algorithm Lectures](#)

質問・誤植報告・補足情報など : [Discord サーバー](#)



AtCoderは、世界最高峰の競技プログラミングサイトです。リアルタイムのオンラインコンテストで競い合うことや、5,000以上の過去問にいつでもチャレンジすることができます。



[ライセンス表記](#)

AtCoderについて

[AtCoderとは？](#)

[競技プログラミングとは？](#)

[レーティングのしくみ](#)

各サービス

[AtCoder](#)

[AtCoderJobs](#)

[アルゴリズム実技検定](#)

[アクセスガイド](#)

[AtCoderCareerDesign](#)

[お知らせ](#)

AtCoder株式会社について

資料請求

[会社概要](#)

[AtCoder](#)

[FAQ](#)

[AtCoderJobs](#)

[プライバシーポリシー](#)

[利用規約](#)